

Module 2

Control Statements and Functions: Control Statements: If Else Statement, Switch Statement, Break Statement, Continue Statement, Looping Structures - While Statement, Do-while Statement, For Statement Functions: Defining A Function, Accessing A Function, Function Prototypes, Passing Arguments To A Function, Return Values And Their Types ,Category Of Functions. Recursion, Storage Classes: Automatic Variables, External, Variables, Static Variables, Register Variable.

Control flow statements in Programming

Control flow refers to the order in which statements within a program execute. While programs typically follow a **sequential flow** from top to bottom, there are scenarios where we need more flexibility. This article provides a clear understanding about everything you need to know about Control Flow Statements.

***Control flow statements** are fundamental components of programming languages that allow developers to control the order in which instructions are executed in a program. They enable execution of a block of code multiple times, execute a block of code based on conditions, terminate or skip the execution of certain lines of code, etc.*

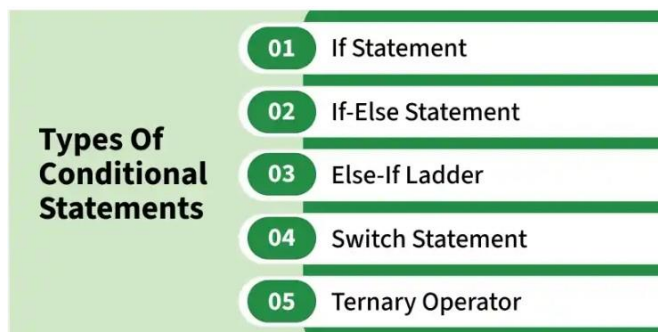
Types of Control Flow statements in Programming:

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is true, and another block if the condition is false.
	switch-case	Evaluates a variable or expression and executes code based on matching cases.
	for	Executes a block of code a specified number of times, typically iterating over a range of values.
Looping Statements	while	Executes a block of code as long as a specified condition is true.
	do-while	Executes a block of code once and then repeats the execution as long as a specified condition is true.

Control Flow Statements Type	Control Flow Statement	Description
Jump Statements	break	Terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch.
	continue	Skips the current iteration of a loop and continues with the next iteration.
	return	Exits a function and returns a value to the caller.
	goto	Transfers control to a labeled statement within the same function. (Note: goto is generally discouraged due to its potential for creating unreadable and error-prone code.)

Conditional Statements in Programming

Conditional statements help a program make decisions. They check whether a condition is true or false and execute different blocks of code based on the result. This allows programs to behave differently in different situations.



Conditional Statements in C,C++,Java,C#,Javascript				
If Statement	If-else Statement	If-else-If Statement	Nested-if Statement	Switch Statement
<pre>if(condition) { // code }</pre>	<pre>if(condition) { // code } else { // code }</pre>	<pre>if(condition) { // code } else if(condition 2) { // code } else { // code }</pre>	<pre>if(condition1) { if(condition2) { // code } else { // code } } else { if(condition3) { // code } else { // code } }</pre>	<pre>switch(expression) { case value1: // code break; case value2: // code break; default: // code }</pre>
Ternary Operator (condition)? true : false				

If Conditional Statement: The if statement checks a condition and executes a block of code only when the condition is true.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 7;
```

```
    if (x > 0)
```

```
    {
```

```
        printf("x is positive");
```

```
    }
```

```
    return 0;
```

```
}
```

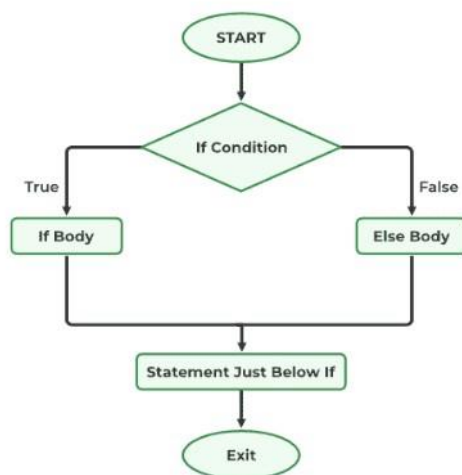
Output

x is positive

If-Else Conditional Statement

The if-else statement checks a condition and runs one block of code if the condition is true, and another block of code if the condition is false.

Flowchart of If-Else Statement



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```

int x = 7;
if (x > 0)
{
    printf("x is positive");
}
else
{
    printf("x is not positive");
}
return 0;
}

```

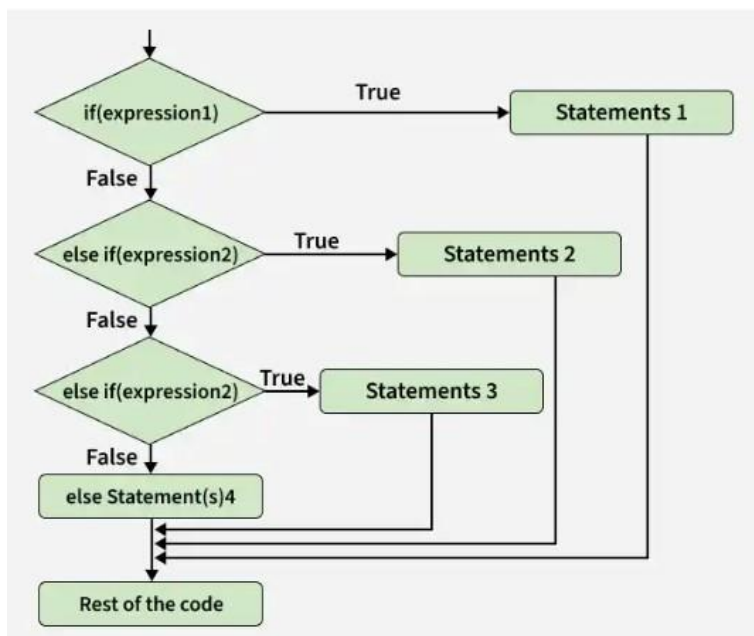
Output

x is positive

if-Else if Conditional Statement

The if-else if statement is used to check multiple conditions. The program evaluates each condition one by one and executes the block of code for the first condition that is true.

Flowchart of If-Else if Statement



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int x = 0;
if (x > 0)
{
    printf("x is positive");
}
else if (x)
{
    printf("x is negative");
}
else
{
    printf("x is zero");
}
return 0;
}
```

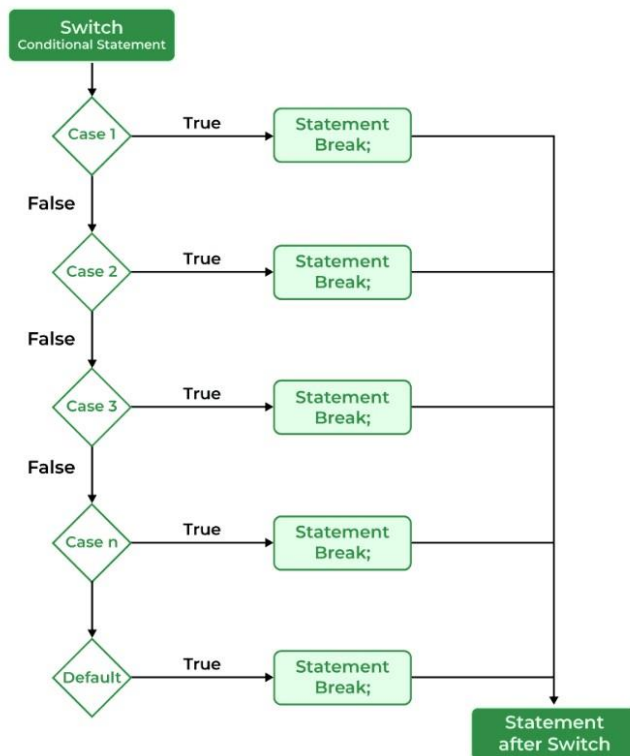
Output

x is zero

Switch Conditional Statement

The switch statement checks a variable against multiple possible values. Each option is written as a case, and the program executes the matching case. A break statement is usually used to stop execution after a case runs.

Flowchart of Switch Statement



Rules of the Switch Statement

When using switch statements, there are a few important rules.

- Case values must be constant values.
- Duplicate case values are not allowed.
- A switch statement can contain multiple case blocks.
- The break statement is optional, but recommended to prevent fall-through.
- The default case is optional but useful for handling unexpected values

```
#include <stdio.h>
```

```
int main() {
```

```
    int day = 2;
```

```
    switch (day) {
```

```
        case 1:
```

```
            printf("Monday\n");
```

```
            break;
```

```
        case 2:
```

```
            printf("Tuesday\n");
```

```
            break;
```

```
        case 3:
```

```

        printf("Wednesday\n");

        break;

default:

    printf("Invalid day!\n");

}

return 0;

}

```

Output

Tuesday

Jump Statements in Programming:

Jump statements in programming are used to change the flow of control within a program. They allow the programmer to transfer program control to different parts of the code based on certain conditions or requirements. Here are common types of jump statements:

1. Break Statement in Programming:

The break statement is primarily used to exit from loops prematurely. When encountered inside a loop, it terminates the loop's execution and transfers control to the statement immediately following the loop.

```

#include <stdio.h>

int main()
{
    for (int i = 0; i < 10; i++) {
        if (i == 5)
            break;

        printf("%d ", i);
    }

    return 0;
}

```

Output

0 1 2 3 4

2. Continue Statement in Programming:

The continue statement is used to skip the current iteration of a loop and proceed to the next iteration.

```

#include <stdio.h>

```

```
int main()
{
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 1)
            continue;
        printf("%d ", i);
    }
    return 0;
}
```

Output

0 2 4 6 8

3. Return Statement in Programming:

The return statement is used to exit a function and optionally return a value to the caller.

```
#include <stdio.h>
```

```
int isEven(int N) { return N % 2 == 0; }
```

```
int main()
{
    int N = 5;
    if (isEven(N)) {
        printf("N is even");
    }
    else {
        printf("N is odd");
    }
    return 0;
}
```

Output

N is odd

4. Goto Statement in Programming:

Some programming languages support the goto statement, which allows transferring control to a labeled statement within the same function or block of code. However, the use of goto is generally discouraged due to its potential for creating unreadable and unmaintainable code.

```
#include <stdio.h>
```

```
int main() {
```

```
    int i = 0;
```

```
loopStart:
```

```
    if (i < 5) {
```

```
        printf("%d ", i);
```

```
        i++;
```

```
        goto loopStart;
```

```
    }
```

```
    return 0;
```

```
}
```

Output

0 1 2 3 4

Loops in C

In C programming, there is often a need for repeating the same part of the code multiple times. For example, to print a text three times, we have to use printf() three times as shown in the code:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf( "Hello GfG\n");
```

```
    printf( "Hello GfG\n");
```

```
    printf( "Hello GfG");
```

```
    return 0;
```

```
}
```

Output

Hello GfG

Hello GfG

Hello GfG

But if we say to write this 20 times, it will take some time to write statement. Now imagine writing it 100 or 1000 times. Then it becomes a really hectic task to write same statements again and again. To solve such kind of problems, we have loops in programming languages.

In the following code, we have used a loop to print text 3 times. We could have done it for 100 or even 1000 times in the same number of code lines.

```
#include <stdio.h>

int main() {

    // Loop to print "Hello GfG" 3 times

    for (int i = 0; i < 3; i++) {

        printf("Hello GfG\n");

    }

    return 0;

}
```

Output

Hello GfG

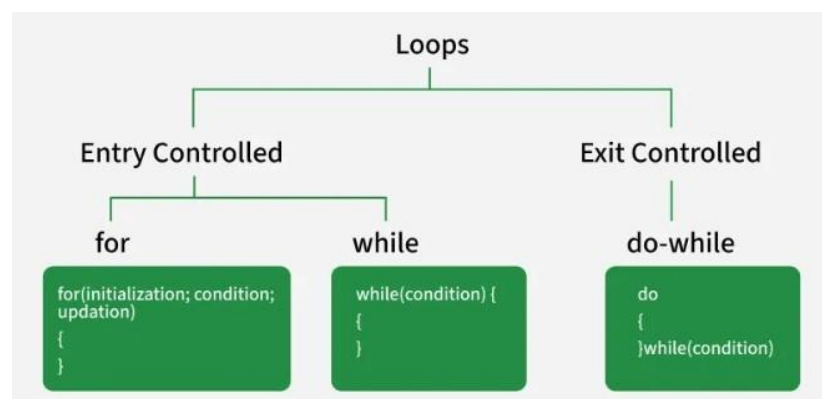
Hello GfG

Hello GfG

Types of loops in C

Loops in C programming are used to repeat a block of code until the specified condition is met. It allows programmers to execute a statement or group of statements multiple times without writing the code again and again.

There are 3 looping statements in C:



for Loop

for loop is an **entry-controlled** loop, which means that the condition is checked before the loop's body executes.

Syntax

```
for (initialization; condition; updation) {  
    // body of for loop  
}
```

The various parts of the for loop are:

- **Initialization:** Initialize the variable to some initial value.
- **Test Condition:** This specifies the test condition. If the condition evaluates to true, then body of the loop is executed. If evaluated false, loop is terminated.
- **Update Expression:** After the execution loop's body, this expression increments/decrements the loop variable by some value.
- **Body of Loop:** Statements to repeat. Generally enclosed inside **{ } braces**.

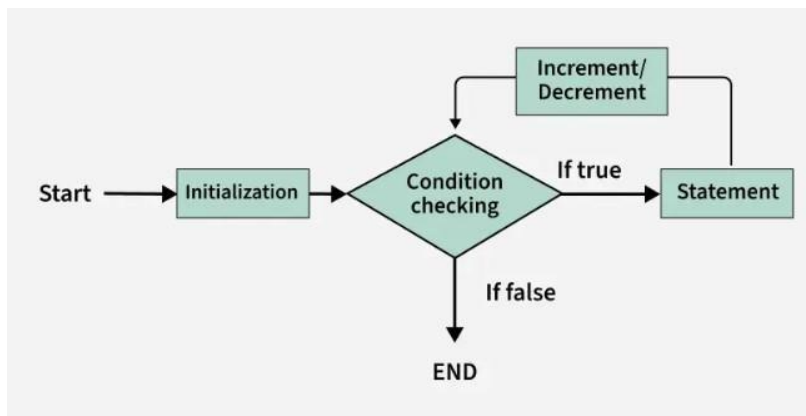
Example:

```
#include <stdio.h>  
  
int main() {  
    // Loop to print numbers from 1 to 5  
    for (int i = 0; i < 5; i++) {  
        printf( "%d ", i + 1);  
    }  
    return 0;  
}
```

Output

1 2 3 4 5

Flowchart of for Loop



while Loop

A **while loop** is also an **entry-controlled loop** in which the condition is checked before entering the body.

Syntax

```
while (condition) {  
    // Body of the loop  
}
```

Only the **condition** is the part of **while loop** syntax, we have to initialize and update loop variable manually.

Example:

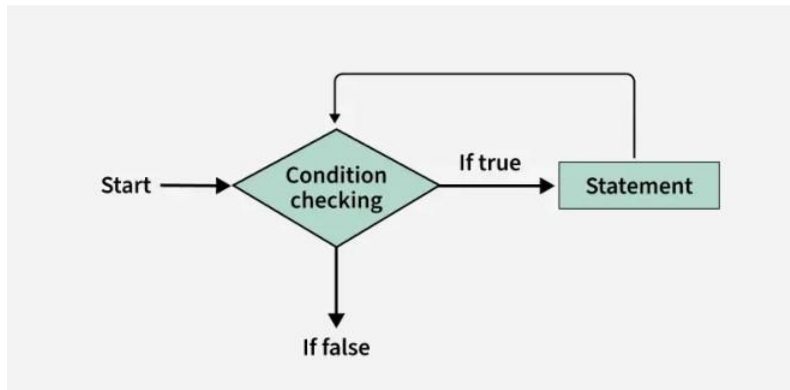
```
#include <stdio.h>  
  
int main() {  
    // Initialization expression  
    int i = 0;  
    // Test expression  
    while(i <= 5) {  
        printf("%d ", i + 1);  
        // update expression  
        i++;  
    }  
    return 0;  
}
```

Output

1 2 3 4 5 6

Flowchart of while Loop

The below flowchart demonstrates execution flow of the **while loop**.



do-while Loop

The do-while loop is an **exit-controlled loop**, which means that the condition is checked after executing the loop body. Due to this, the loop body will **execute at least once** irrespective of the test condition.

Syntax

```
do {  
    // Body of the loop  
} while (condition);
```

Like while loop, only the condition is the part of **do while loop** syntax, we have to do the initialization and updating of loop variable manually.

Example:

```
#include <stdio.h>  
  
int main() {  
    // Initialization expression  
    int i = 0;  
  
    do  
    {  
        // loop body  
        printf( "%d ", i);  
  
        // Update expression  
        i++;  
  
        // Condition to check
```

```

} while (i <= 10);

return 0;

}

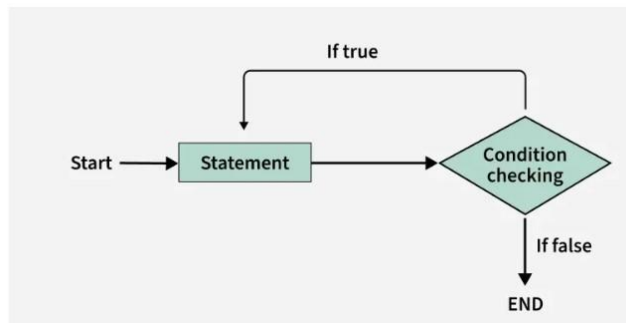
```

Output

0 1 2 3 4 5 6 7 8 9 10

Flowchart of do-while Loop

The below flowchart demonstrates execution flow of the do while loop.



Functions

One of the important strengths of the **C programming language** is its support for **functions**. Functions make a program easier to write, understand, test, debug, and reuse.

A **function** is a self-contained block of statements designed to perform a particular task.

```
#include <stdio.h>
```

```

int main()
{
    printf("Hello World");
    return 0;
}

```

In this program:

- main() is a function.
- printf() is a function provided by the C library.

Similarly, scanf() is also a function.

A function is a named block of code that performs a specific task. A function can take inputs (called parameters), execute a block of statements, and optionally return a result.

- A function allows you to write a piece of logic once and reuse it wherever needed in the program.

- This helps keep your code clean, organized, easier to understand and manage.

```
#include <stdio.h>

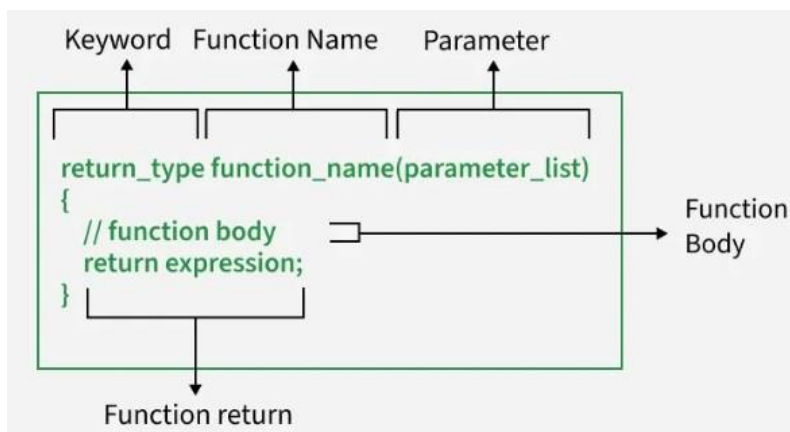
// function definition
int square(int x)
{
    return x * x;
}

int main()
{
    // Calling the function
    int result = square(5);
    printf("Square of 5 is: %d", result);
    return 0;
}
```

Output

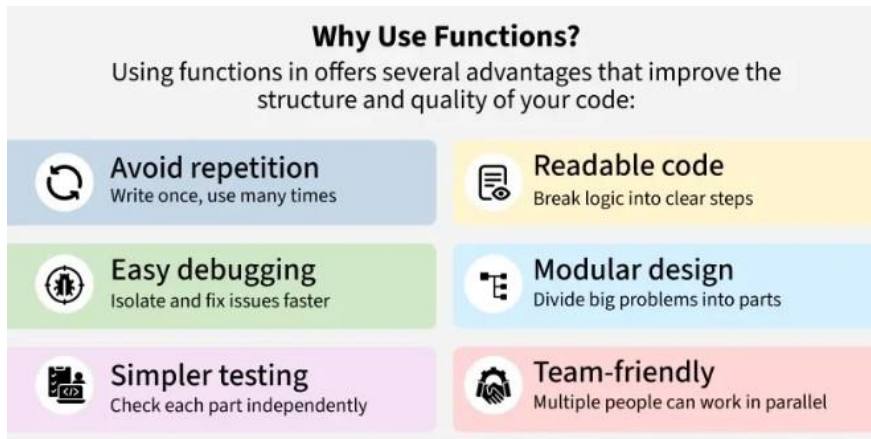
Square of 5 is: 25

Syntax



- **Return type:** Specifies the type of value the function will return. Use void if the function does not return anything.
- **Function name:** A unique name that identifies the function. It follows the same naming rules as variables.
- **Parameter list:** A set of input values passed to the function. If the function takes no inputs, this can be left empty or written as void.

- **Function body:** The block of code that runs when the function is called. It is enclosed in curly braces { }.



Functions provide several advantages in program development.

1 Modularity

A large program can be divided into smaller, manageable functions.

For example:

Student Management System

```
|
|— addStudent()
|— displayStudent()
|— searchStudent()
|— deleteStudent()
```

Each function performs a specific task.

2 Code Reusability

A function can be called multiple times from different parts of a program.

```
display();
```

```
display();
```

```
display();
```

The same code does not have to be written three times.

3 Easy Debugging

If an error occurs in a particular task, we can examine the corresponding function instead of checking the entire program.

4 .Easy Maintenance

Functions make programs easier to modify and maintain.

For example, if the calculation of salary changes, we can modify the salary calculation function without changing the entire program.

5 Better Readability

Dividing a large program into meaningful functions makes the program easier to understand.

Function Declaration Vs Definition

It's important to understand the difference between declaring a function and defining it. Both play different roles in how the compiler understands and uses your function.

Function Declaration

A declaration tells the compiler about the function's name, return type, and parameters before it is actually used. It does not contain the function's body. This is often placed at the top of the program or in a header file.

```
int add(int a, int b);
```

Function Definition

A definition provides the actual implementation of the function. It includes the full code or logic that runs when the function is called.

```
int add(int a, int b) {  
    return a + b;  
}
```

Calling a Function

Once a function is defined, you can use it by simply calling its name followed by parentheses. This tells the program to execute the code inside that function.

```
#include <stdio.h>
```

```
// Function definition
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {
```

```
    // Function call
```

```
    int result = add(5, 3);
```

```
    printf("The sum is: %d", result);
```

```
    return 0;
}
```

Output

The sum is: 8

Explanation

- The `add()` function is called with the values 5 and 3, computes their sum, and returns the result, which is stored in the variable `result`.
- A function can be called multiple times from `main()` or other functions, allowing the same code to be reused whenever needed.
- Function calls improve code reusability, reduce duplication, and make programs easier to read, maintain, and debug.

Types of Function in C

In C programming, functions can be grouped into two main categories: **library functions** and **user-defined functions**. Based on how they handle input and output, user-defined functions can be further classified into different types.

Library Functions

These are built-in functions provided by C, such as `printf()`, `scanf()`, `sqrt()`, and many others. You can use them by including the appropriate header file, like `#include <stdio.h>` or `#include <math.h>`.

User-Defined Functions

These are functions that you create yourself to perform specific tasks in your program. Depending on whether they take input or return a value, they can be of four types:

- **No arguments, no return value:** The function neither takes input nor returns any result.
- **Arguments, no return value:** The function takes input but does not return anything.
- **No arguments, return value:** The function does not take input but returns a result.
- **Arguments and return value:** The function takes input and returns a result.

Each type serves different purposes depending on what the program needs. Using the right type helps make your code more organized and efficient.

Memory Management of Functions

When a function is called, the system allocates memory for its parameters and local variables in a stack frame, which is automatically released when the function finishes execution.

- **Automatic Memory Allocation:** Each function call creates a separate stack frame to store its local variables, parameters, and return information.

- **Automatic Memory Deallocation:** Once the function returns, its stack frame is removed from the call stack, freeing the allocated memory automatically.

Function Prototypes

A **function prototype** is a **declaration of a function** that tells the compiler **about the function before the function is actually called**.

Example

```
int add(int, int);
```

This is a function prototype.

It tells the compiler:

- add → function name
- int before add → function returns an integer
- (int, int) → function accepts two integer arguments

It does **not** contain the actual implementation.

```
#include <stdio.h>
```

```
int add(int, int); // Function prototype
```

```
int main()
```

```
{
```

```
    int result;
```

```
    result = add(10, 20);
```

```
    printf("%d", result);
```

```
    return 0;
```

```
}
```

```
int add(int a, int b) // Function definition
```

```
{
```

```
    return a + b;
```

```
}
```

Two Main Parts of a Function Definition

A function definition has two major parts:

Function Definition

1. Function Header

2.Function Body

Function Header

The function header contains:

1. Return type
2. Function name
3. Formal parameter list

Example:

```
int sum(int a, int b)
```

Function Body

The function body is enclosed within { }.

It contains:

- Local declarations
- Executable statements
- Return statement, when required

Example:

```
{  
    int result;  
    result = a + b;  
    return result;  
}
```

Function Header

Consider:

```
float mul(float x, float y)
```

The three components are:

float	mul	(float x, float y)
↓	↓	↓
Return	Function	Formal
type	name	parameters

Function Type / Return Type

The **return type** specifies the type of value returned by the function.

Examples:

1.int sum(int a, int b)

returns an int.

2.float average(float a, float b)

returns a float.

3.double power(double x, int n)

returns a double.

void Return Type

When a function does not return any value, its return type should be void.

Example:

```
void printMessage(void)
{
    printf("Welcome");
}
```

Here:

- First void → function does not return a value.
- Second void → function does not accept parameters.

If a function does not return a value, use:

void

Example:

```
void display(void)
{
    printf("Hello");
}
```

Function Name

A function name is an identifier.

Therefore, it must follow the rules of C identifiers.

Examples:

sum()

calculateTotal()

DisplayResult()

findMaximum()

A meaningful function name should describe the task performed by the function.

For example:

calculateAverage()

is better than:

fun1()

It is also advisable to avoid names that conflict with library functions or other identifiers.

Formal parameters are variables declared in the function header that receive values from the calling function.

Example:

```
int sum(int a, int b){  
}
```

Here:

a and b → Formal parameters

When the function is called:

sum(10, 20);

the values are passed to the parameters.

10 ———→ a

20 ———→ b

Then:

return a + b;

produces:

30

Actual Parameters

Actual parameters are the values or variables supplied to a function when the function is called.

They are also commonly called actual arguments.

Example

```
int add(int a, int b)
```

```
{  
    return a + b;  
}
```

```
int main()
```

```
{  
    int result;  
    result = add(10, 20);  
}
```

Here:

```
add(10, 20);
```

- 10 → actual parameter
- 20 → actual parameter
- a and b → formal parameters

Another example

```
int multiply(int x, int y)
```

```
{  
    return x * y;  
}
```

```
int main()
```

```
{  
    int a = 5, b = 4;  
    multiply(a, b);  
}
```

Here:

- a and b → actual parameters
- x and y → formal parameters

Consider:

```
int sum(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
int main()
```

```
{
```

```
    sum(10, 20);
```

```
}
```

Function Without Parameters

A function does not always need to receive data from the calling function.

Example:

```
void printline(void)
```

```
{
```

```
    printf("-----");
```

```
}
```

Here, the function has **no parameters**.

The keyword `void` inside the parentheses explicitly indicates that the function accepts no arguments.

Calling it:

```
printline();
```

Function Body

The function body is enclosed within braces:

```
{
```

```
    ...
```

```
}
```

It generally contains three components:

1. Local declarations

Variables required by the function are declared here.

```
int i, sum;
```

2. Function statements

These statements perform the required task.

```
sum = a + b;
```

3. Return statement

The result can be returned to the calling function.

```
return sum;
```

Return Values And Their Types

A function may or may not return a value to the **calling function**. When a function needs to send a result back, it uses the **return statement**.

A function can receive any number of input values through parameters, but **one function call can return at most one value**.

1. Syntax of the return Statement

There are two basic forms:

```
return;
```

and

```
return expression;
```

2. Plain return Statement

The statement

```
return;
```

does **not return a value**. It immediately terminates the execution of the current function and transfers control back to the calling function.

Example

```
void check(int error)
{
    if(error)
        return;
    printf("No error");
}
```

If error is non-zero, the return statement is executed and the function ends immediately.

3. return with an Expression

The second form is:

return expression;

The expression is evaluated, and its value is sent back to the calling function.

Example

```
int multiply(int x, int y)
{
    int p;
    p = x * y;
    return p;
}
```

If we call:

```
int result;
result = multiply(5, 4);
```

The function calculates:

$p = 5 \times 4$

$p = 20$

and:

return p;

returns 20 to the calling function.

4. Direct Return of an Expression

The following:

```
p = x * y;
```

```
return p;
```

can be simplified to:

```
return (x * y);
```

Complete example:

```
int multiply(int x, int y)
{
    return (x * y);
}
```

This is more concise because the calculated result is returned directly.

5. A Function Can Have Multiple return Statements

A function can contain more than one return statement.

This is useful when the value to be returned depends on a condition.

Example

```
int maximum(int a, int b)
```

```
{  
    if(a > b)  
        return a;  
    else  
        return b;  
}
```

Here, there are two possible return statements:

a > b

```
|  
├── Yes → return a  
|  
└── No → return b
```

However, **only one of them will be executed during a particular function call.**

CATEGORY OF FUNCTIONS

A function, depending on whether arguments are present or not and whether a value is returned or not, may belong to one of the following categories:

Category 1: Functions with no arguments and no return values.

Category 2: Functions with arguments and no return values.

Category 3: Functions with arguments and one return value.

Category 4: Functions with no arguments but return a value.

Category 5: Functions that return multiple values.

1. Function with No Arguments and No Return Value

This type of function:

- Does not receive any data from the calling function.
- Does not return any value.
- Performs its task independently.

Example

```
#include <stdio.h>

void display(void)
{
    printf("Welcome to C Programming");
}

int main()
{
    display();
    return 0;
}
```

Here:

```
void display(void)
```

- First void → no return value
- Second void → no arguments

2. Function with No Arguments but With Return Value

This type of function:

- Does not receive arguments.
- Performs some operation.
- Returns a value to the calling function.

Example

```
#include <stdio.h>

int getNumber(void)
{
    int n;
    printf("Enter a number: ");
```

```

    scanf("%d", &n);
    return n;
}

int main()
{
    int number;
    number = getNumber();
    printf("Number = %d", number);
    return 0;
}

```

Here:

```
int getNumber(void)
```

- void → no arguments
- int → returns an integer

The returned value is received by:

```
number = getNumber();
```

3. Function with Arguments but No Return Value

This type of function:

- Receives data from the calling function.
- Performs an operation.
- Does not return a value.

Example

```

#include <stdio.h>

void displaySum(int a, int b)
{
    int sum;
    sum = a + b;
    printf("Sum = %d", sum);
}

```

```
int main()
{
    displaySum(10, 20);
    return 0;
}
```

Here:

```
displaySum(10, 20);
```

10 and 20 are **actual arguments**.

They are received by:

```
void displaySum(int a, int b)
```

where a and b are **formal parameters**.

The function displays the result but does not return it.

4. Function with Arguments and With Return Value

This is one of the most commonly used forms.

The function:

- Receives data through arguments.
- Performs an operation.
- Returns a value to the calling function.

Example

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main()
{
    int result;
    result = add(10, 20);
    printf("Sum = %d", result);
}
```

```
    return 0;  
}
```

Here:

10, 20 → Actual arguments

a, b → Formal parameters

30 → Return value

Storage Classes

A variable is a named memory location used to store data that may change during program execution.

A variable in C has:

1. Data type – specifies what type of data it stores.
2. Storage class – specifies its scope, lifetime, and visibility.

For example:

```
int count;
```

Here:

- int → data type
- count → variable name

A storage class can also be specified:

```
static int count;
```

Here, static is the storage class specifier.

Scope of a Variable

Scope refers to the portion of a program where a variable can be accessed or recognized.

Variables are broadly classified as:

- Global variables
- Local variables

Global Variables

A variable declared outside all functions is called a global variable.

Example:

```
#include <stdio.h>
```

```
int m = 10;
```

```

void display()
{
    printf("%d", m);
}

int main()
{
    printf("%d", m);
    display();
    return 0;
}

```

Here:

```
int m = 10;
```

is a global variable.

It can be accessed by functions in the same source file, subject to normal scope/linkage rules.

Global variables are also commonly referred to as external variables.

Local Variables

A variable declared inside a function is called a local variable.

Example:

```

void calculate()
{
    int sum;
    int balance;
}

```

Here:

```
sum
```

```
balance
```

are local variables.

They can be accessed only within the function/block where they are declared.

Example

```
#include <stdio.h>
```

```
void function1()
```

```
{
```

```
    int i = 10;
```

```
    printf("%d", i);
```

```
}
```

```
void function2()
```

```
{
```

```
    int i = 20;
```

```
    printf("%d", i);
```

```
}
```

Although both functions have a variable named `i`, they are different variables.

Changing `i` in `function1()` does not change `i` in `function2()`.

A storage class in C specifies the scope, lifetime, visibility, and storage location of a variable.

In simple words, storage class tells us:

- Where a variable can be accessed
- How long the variable exists
- What its default value is
- Whether it can be accessed from other functions/files

The main storage classes in C are:

1. `auto` – Automatic variables
2. `extern` – External variables
3. `static` – Static variables
4. `register` – Register variables

Examples

```
auto int count;  
register char ch;  
static int x;  
extern long total;
```

Storage class	Meaning
auto	Local variable known only to the function in which it is declared. <i>Default is auto.</i>
static	Local variable which exists and retains its value even after the control is transferred to the calling function.
extern	Global variables known to all functions in the file and to functions in other files which have been linked to this file.
register	Local variable which is requested to be stored in a CPU register subject to availability by the programmer.

Yes. The previous version had some duplication. Here is a **properly arranged, concise PG-level note** covering the four storage classes without repeating concepts.

Storage Classes in C

Storage classes in C determine the **scope, lifetime, visibility, and storage behavior** of variables and functions.

The major storage classes are:

1. **auto** – Automatic variables
2. **extern** – External variables
3. **static** – Static variables
4. **register** – Register variables

Automatic Variables

An **automatic variable** is a local variable that is created when the function in which it is declared is called and is automatically destroyed when that function finishes execution.

Automatic variables are mainly associated with **function-level local storage**.

Declaration

A variable declared inside a function **without specifying a storage class** is automatically treated as an automatic variable.

```
void display()
{
    int number = 10;
}
```

Here, number is an automatic variable by default.

The auto keyword can also be used explicitly:

```
void display()
{
```

```
    auto int number = 10;
}
```

Both declarations represent automatic variables.

Syntax

```
auto data_type variable_name;
```

Example:

```
auto int count;
```

Scope

The scope of an automatic variable is limited to the **block or function** in which it is declared.

```
void function1()
```

```
{
    int x = 10;
    printf("%d", x);
}
```

Here, x can be accessed only inside function1().

Automatic variables are therefore also called:

- Local variables
- Internal variables

Lifetime

The lifetime of an automatic variable is associated with the execution of its block or function.

When a function is called:

1. Memory is allocated for the variable.
2. The variable becomes active.
3. The function executes.
4. When the function terminates, the variable is destroyed.

Same Variable Name in Different Functions

Different functions can have automatic variables with the same name because each variable has its own local scope.

```
#include <stdio.h>
```

```
void function1()
```

```

{
    int m = 10;
    printf("function1: %d\n", m);
}

void function2()
{
    int m = 100;
    printf("function2: %d\n", m);
}

int main()
{
    int m = 1000;
    function1();
    function2();
    printf("main: %d\n", m);
    return 0;
}

```

Here, the three variables named m are separate variables.

External Variables

An **external variable** is a variable declared outside all functions. It is also called a **global variable**.

An external variable normally remains alive throughout the execution of the program and can be accessed by multiple functions where it is visible.

Declaration

External variables are declared outside functions.

```
int number;
```

```
float length;
```

Complete Example

```
#include <stdio.h>
```

```
int number;
```

```

float length;

void function1()
{
    printf("Number = %d\n", number);
}

void function2()
{
    printf("Length = %.2f\n", length);
}

int main()
{
    number = 10;
    length = 25.5;
    function1();
    function2();
    return 0;
}

```

Here, number and length are external/global variables.

Characteristics

- Declared outside functions.
- Normally remain alive throughout program execution.
- Can be accessed by multiple functions where visible.
- Their values can be modified by functions that have access to them.
- They can be used to share data between functions.
- They do not need to be passed as function arguments when directly accessible.

Local Variable Hiding a Global Variable

If a local variable and a global variable have the same name, the **local variable takes precedence** within its scope.

```
#include <stdio.h>
```

```
int count = 10;
```

```

void fun()
{
    int count = 20;
    printf("count = %d\n", count);
}

int main()
{
    fun();
    printf("count = %d\n", count);
    return 0;
}

```

Output

count = 20

count = 10

The local count inside fun() hides the global count.

External Declaration – extern

A global variable is available from its **point of declaration** to the end of the source file.

If a variable is defined after the function that uses it, an **external declaration** can be used.

Consider:

```
#include <stdio.h>
```

```
int main()
```

```
{
    printf("%d", y);
    return 0;
}
```

```
int y = 10;
```

Here, y has not been declared when main() uses it.

This can be solved using extern.

```
#include <stdio.h>
```

```
extern int y;

int main()
{
    printf("%d", y);
    return 0;
}

int y = 10;
```

Here:

```
extern int y;
```

informs the compiler that an integer variable y is defined somewhere else.

Definition and Declaration

Definition

```
int y = 10;
```

A definition:

- Defines the variable.
- Allocates storage for the variable.
- Specifies its type and initial value.

External Declaration

```
extern int y;
```

An external declaration:

- Informs the compiler that y exists.
- Provides type information.
- Does **not** allocate storage for y.

extern with Multiple Source Files

extern is particularly useful when a global variable is defined in one source file and used in another.

file1.c

```
int count = 100;
```

file2.c

```
#include <stdio.h>
```

```
extern int count;

void display()
{
    printf("%d", count);
}
```

Here, count is defined in file1.c and declared using extern in file2.c.

Static Variables

A variable can be declared using the static keyword:

```
static int x;
```

A static variable has **static storage duration**, meaning that its value persists throughout program execution.

A static variable can be:

1. Local static variable
2. External static variable

Local Static Variables

A local static variable is declared inside a function.

Unlike an automatic variable, a local static variable **retains its value between function calls**.

Example:

```
#include <stdio.h>

void stat()
{
    static int x = 0;

    x++;

    printf("x = %d\n", x);
}
```

```
int main()
{
    stat();

    stat();
}
```

```
    stat();  
    return 0;  
}
```

Output

x = 1

x = 2

x = 3

The variable x is initialized only once.

During the first call:

$x = 0 \rightarrow x++ \rightarrow 1$

During the second call:

$x = 1 \rightarrow x++ \rightarrow 2$

During the third call:

$x = 2 \rightarrow x++ \rightarrow 3$

Therefore, the value of x is preserved between function calls.

Automatic vs Static Local Variable

If x were automatic:

```
void stat()  
{  
    int x = 0;  
    x++;  
    printf("x = %d\n", x);  
}
```

Calling the function three times would produce:

x = 1

x = 1

x = 1

because the automatic variable is created again for each function call.

External Static Variables

A static variable declared outside all functions is called an **external static variable**.

```
static int x = 100;
```

It has a lifetime throughout program execution but is accessible only within the **source file in which it is defined**.

This is different from an ordinary global variable:

```
int x = 100;
```

An ordinary global variable can be accessed from another source file using an appropriate extern declaration.

A file-scope static variable cannot be accessed directly from another source file.

Static Functions

The static keyword can also be applied to functions.

```
static void display()
{
    printf("Hello");
}
```

A static function can be accessed only within the source file in which it is defined.

Register Variables

A register variable is declared using the register keyword.

```
register int count;
```

The keyword requests the compiler to keep the variable in a **CPU register** for faster access.

Register variables are useful conceptually for frequently accessed variables such as loop-control variables.

Example

```
#include <stdio.h>

int main()
{
    register int count;
    for (count = 1; count <= 5; count++)
    {
        printf("%d\n", count);
    }
}
```

```

    }
    return 0;
}

```

Here, count is declared as a register variable.

Comparison of Storage Classes

Storage Class	Declaration	Scope	Lifetime	Main Purpose
auto	Inside block/function	Local	During block execution	Temporary local storage
extern	Outside or inside function	Depends on declaration	Normally entire program	Refer to a variable defined elsewhere
static	Inside or outside function	Local or file-level	Entire program	Preserve value / restrict file visibility
register	Inside block/function	Local	During block execution	Request register storage